



1. 实数的表示

1.1 浮点数的表示格式



$$N = (-1)^s \times M \times R^E$$

s取值为0或1，用来决定数x的符号

M是一个二进制定点小数，称为数x的尾数

一般用**原码**表示

E是一个二进制定点整数，称为数x的阶或指数

一般用**移码**表示

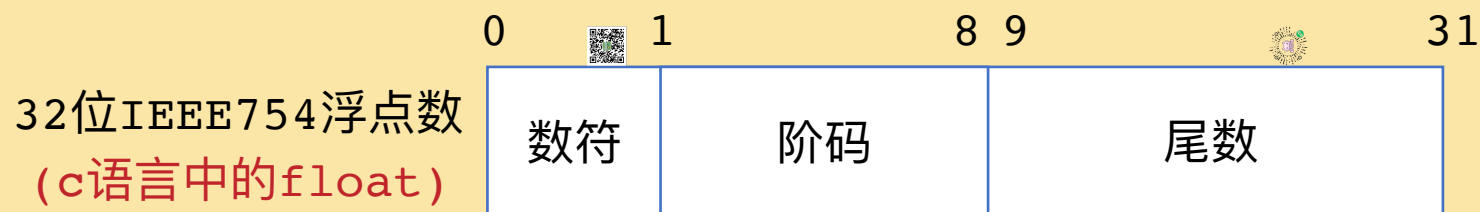
R是基数，可以约定为2、4、16等



1. 实数的表示

1.2 IEEE754标准

float的23位尾数实际上提供了24位有效数字
double的52位尾数实际上提供了53位有效数字



真值

$$(-1)^s \times 1.f \times 2^{e-127}$$

$$(-1)^s \times 1.f \times 2^{e-1023}$$

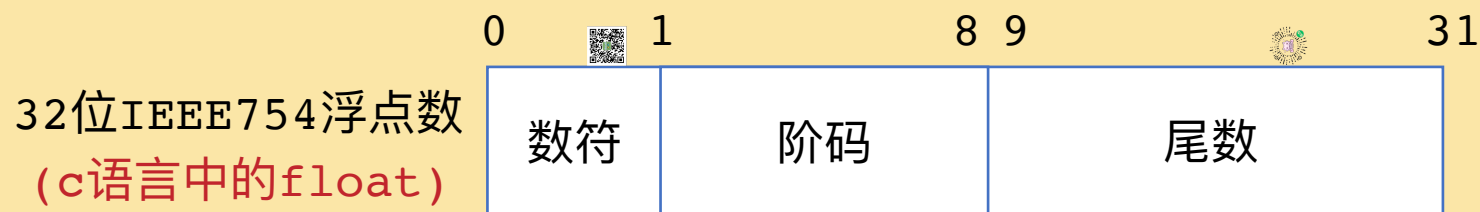
IEEE754采用隐藏位策略
小数点前固定为1，并且这个1不会存储



1. 实数的表示

1.2 IEEE754标准

IEEE754的阶码采用移码表示
但偏执值不是n位移码的所用的 2^{n-1}
而是 $2^{n-1}-1$



真值

$$(-1)^s \times 1.f \times 2^{e-127}$$

$$(-1)^s \times 1.f \times 2^{e-1023}$$

float偏置值为127，E的取值为1~254

double偏置值为1023，E的取值为1~2046

float偏置值为127，E的取值为1~254
double偏置值为1023，E的取值为1~2046



1. 实数的表示

1.2 IEEE754标准

值的类型	符号	阶码	尾数	值
正零	0	0	0	0
负零	1	0	0	0
正无穷大	0	255/2047	0	∞
负无穷大	1	255/2047	0	$-\infty$
无定义数	-	255/2047	$\neq 0$	NaN (Not a Number)
非规格化数	-	0	f	$2^{-126} \times 0.f$ $2^{-1022} \times 0.f$



float偏置值为127，E的取值为1~254
double偏置值为1023，E的取值为1~2046

1. 实数的表示

1.2 IEEE754标准

值的类型	符号	阶码	尾数	值
非规格化数	-	0	f	$2^{-126} \times 0.f$ $2^{-1022} \times 0.f$

已知float 型变量用IEEE 754 单精度浮点数格式表示。若float型变量x 的机器数为8020 0000H，则x的值是（ ）。

- A. -2^{-128}

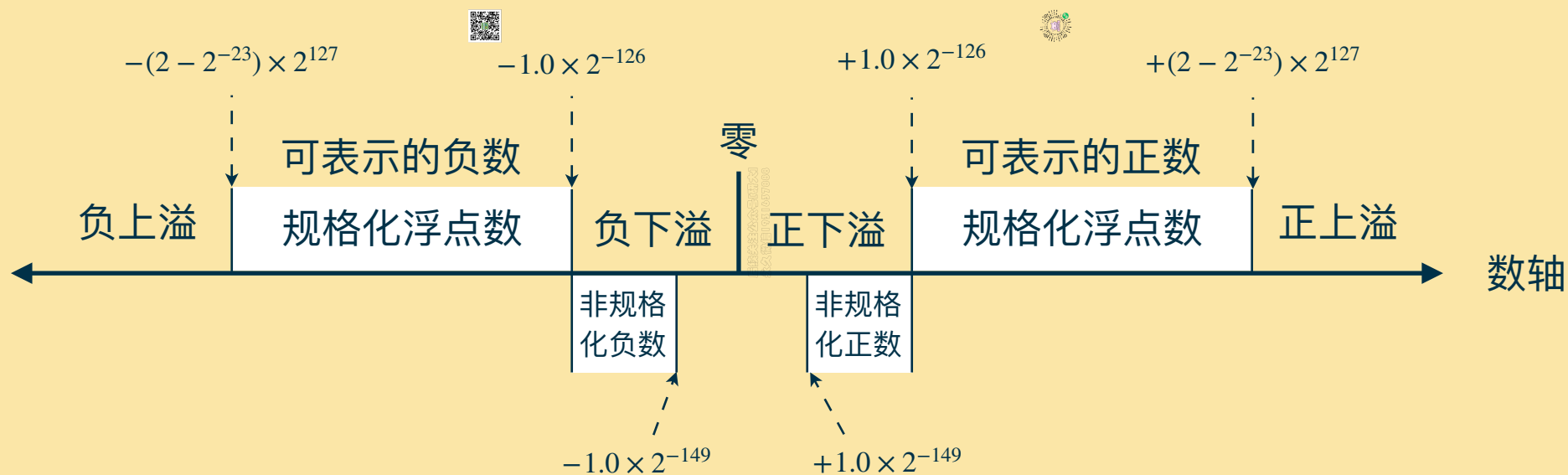
B. -1.01×2^{-127}
- C. -1.01×2^{-126}

D. 非数(NaN)



1. 实数的表示

1.2 IEEE754标准



IEEE754 标准的单精度浮点数的表示范围



1. 实数的表示

1.2 IEEE754标准

将十进制数 -0.75 转换为 IEEE754 的单精度浮点数格式表示



$$-0.75 = -1.1 \times 2^{-1} = (-1)^s \times 1.f \times 2^{e-127}$$

$$s=1$$

$$f=1$$

$$e=126 = 0111\ 1110_{(二进制)}$$

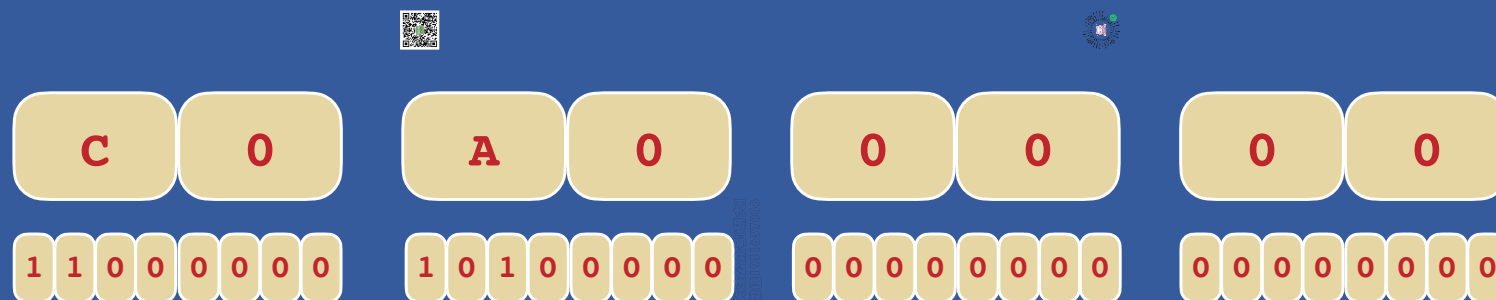
浮点数格式: 1 0 1 1 1 1 1 0 1 0 0 0 0 0 0 0



1. 实数的表示

1.2 IEEE754标准

求 IEEE754 单精度浮点数 C0 A0 00 00H 的值是多少



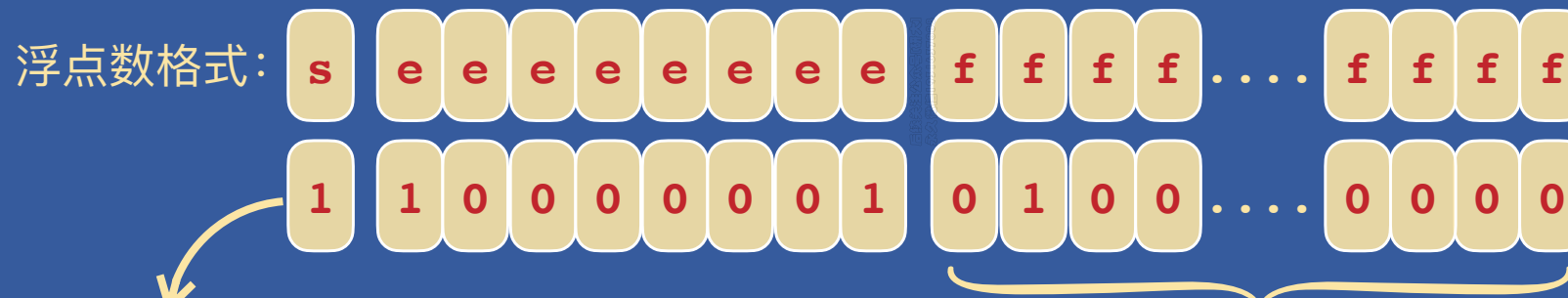
浮点数格式: s e e e e e e e f f f f f f f f



1. 实数的表示

1.2 IEEE754标准

求 IEEE754 单精度浮点数 C0 A0 00 00H 的值是多少



是个负数

$$=-1.25 * 2^2 = -5$$



1. 实数的表示

1.3 C语言中的浮点数类型



int→float: 不会发生溢出，可能有数据被舍入

int/float→double: 能保留精确值

double→float: 可能发生溢出，可能被舍入

float/double→int: 向0方向被截断



1. 实数的表示

1.3 C语言中的浮点数类型

假定变量i、f、d的类型分别是int、float 和double，它们可以取除 $+\infty$ 、 $-\infty$ 和NaN以外的任意值。

请判断下列每个C语言关系表达式在32位机器上运行时是否永真。

A. `i == (int) (float) i` 反例: `i=1234567890, (int)(float)i=1234567936`

B. `f == (float) (int) f`

C. `i == (int) (double) i`

D. `f == (float) (double) f`

E. `d == (float) d`

F. `f == -(-f)`

G. `(d+f)-d == f`



1. 实数的表示

1.3 C语言中的浮点数类型

假定变量i、f、d的类型分别是int、float 和double，它们可以取除 $+\infty$ 、 $-\infty$ 和NaN以外的任意值。

请判断下列每个C语言关系表达式在32位机器上运行时是否永真。

A. `i == (int) (float) i` 反例: `i=1234567890, (int)(float)i=1234567936`

B. `f == (float) (int) f` 反例: `f=3.1, (float)(int)f=3`

C. `i == (int) (double) i` 正确

D. `f == (float) (double) f` 正确

E. `d == (float) d` 反例: `d=16777217, (float)d=16777216`

F. `f == -(-f)` 正确

G. `(d+f)-d == f` 反例: $d = 1.79 \times 10^{308}, f = 1.0$ 时, 式子变为 $0=1$



2. 浮点数的加减运算

【例】用IEEE754单精度浮点数加减运算计算0.5+ (-0.4325)



$$x = 0.5 = 0.100\dots0 = 1.00\dots0 \times 2^{-1}$$

$$y = -0.4325 = -0.01110\dots0 = -1.110\dots0 \times 2^{-2}$$



2. 浮点数的加减运算

2.1 对阶

对阶：通过尾数移位使操作数的小数点位置对齐，从而表现出阶码相等

【例】用IEEE754单精度浮点数加减运算计算 $0.5 + (-0.4375)$

$$x = 0.5 = 0.100\dots0 = 1.00\dots0 \times 2^{-1}$$

$$y = -0.4375 = -0.01110\dots0 = -1.110\dots0 \times 2^{-2}$$

小阶向大阶看齐，将阶码小的数尾数右移，每移1位阶码+1

x的阶码为-1，y的阶码为-2
y的阶码更小，二者阶码差1
所以y的尾数需要右移1位，高位补0
注意，隐藏位的1也参与右移

对阶时如果有有效位被移出
一定要保留

后续舍入操作需要参考这些位
但现在移出的是0，所以可以忽略

x

$$+1.00\dots0 \times 2^{-1}$$

y

$$-0.1110\dots(0) \times 2^{-1}$$



2. 浮点数的加减运算

2.2 尾数加减

尾数加减：将对阶后的尾数按定点数加减运算规则进行运算

【例】用IEEE754单精度浮点数加减运算计算 $0.5 + (-0.4325)$

$$x = 0.5 = 0.100\dots0 = 1.00\dots0 \times 2^{-1}$$

$$y = -0.4375 = -0.01110\dots0 = -1.110\dots0 \times 2^{-2}$$

x

$$+1.00\dots0 \times 2^{-1}$$

y

$$-0.1110\dots0 \times 2^{-1}$$

$$M = 1.00\dots0 + (-0.1110\dots0) = 1.00\dots0 - 0.1110\dots0 = 0.0010\dots0$$



2. 浮点数的加减运算

2.3 尾数规格化

规格化： 上一步的尾数求和结束后可能得到不符合规格化的结果，此时要进行尾数左规或右规调整



右右规：

- 当结果为 $\pm 1.x \dots x$ 时，需要进行右规
- 尾数右移1位，阶码+1

左左规：

- 当结果为 $\pm 0.0 \dots 01x \dots x$ 时，需要进行左规
- 尾数每左移1位，阶码-1，直到小数点前为1



2. 浮点数的加减运算

2.3 尾数规格化

规格化： 上一步的尾数求和结束后可能得到不符合规格化的结果，此时要进行尾数左规或右规调整

【例】 用IEEE754单精度浮点数加减运算计算 $0.5 + (-0.4325)$

$$x = 0.5 = 0.100\dots0 = 1.00\dots0 \times 2^{-1}$$

$$y = -0.4375 = -0.01110\dots0 = -1.110\dots0 \times 2^{-2}$$

$x+y$

$$+1.0000\dots0 \times 2^{-4}$$

此时应该左规3次

左规：

- 当结果为 $\pm 1x.x\dots x$ 时，需要进行右规
- 尾数右移1位，阶码+1

右规：

- 当结果为 $\pm 0.0\dots01x\dots x$ 时，需要进行左规
- 尾数每左移1位，阶码-1，直到小数点前为1



2. 浮点数的加减运算

2.4 尾数的舍入处理

舍入：在对阶和尾数右规时，可能会对尾数进行右移，为了保证运算精度，一般将低位移出的保留下来，参加中间过程的运算，最后再将运算结果进行舍入，还原表示为IEEE754格式

就近舍入

朝 $+\infty$ 方向舍入

朝 $-\infty$ 方向舍入

朝0方向舍入



2. 浮点数的加减运算

2.4 尾数的舍入处理

舍入：在对阶和尾数右规时，可能会对尾数进行右移，为了保证运算精度，一般将低位移出的为保留下来，参加中间过程的运算，最后再将运算结果进行舍入，还原表示为IEEE754格式

朝 $+\infty$ 方向舍入

总是取右边最近可表示数，也称为正向舍入或朝上舍入

朝 $-\infty$ 方向舍入

总是取左边最近可表示数，也称为负向舍入或朝下舍入

朝0方向舍入

直接截取所需位数，丢弃后面所有位



2. 浮点数的加减运算

2.5 溢出判断



尾数溢出时可以通过右规进行调整，所以尾数溢出结果不一定溢出

左规/右规可能导致阶减小/增大，从而引发指数下溢/上溢，此时判定浮点数发生溢出，产生异常/按机器零处理

浮点数的溢出不是通过尾数溢出判断的，而是通过指数溢出判断的



3. 数据的宽度和存储

3.1 大小端

大端存储：和人类阅读顺序一致，数据低位放在高地址，数据高位放在低地址

x = 12345678H

数据高位 数据低位

	地址低位			地址高位
地址	0800H	0801H	0802H	0803H
数据	12H	34H	56H	78H



3. 数据的宽度和存储

3.1 大小端

小端存储：和人类阅读顺序相反，数据低位放在低地址，数据高位放在高地址

x = 12345678H

数据高位 数据低位

	地址低位			地址高位
地址	0800H	0801H	0802H	0803H
数据	78H	56H	34H	12H



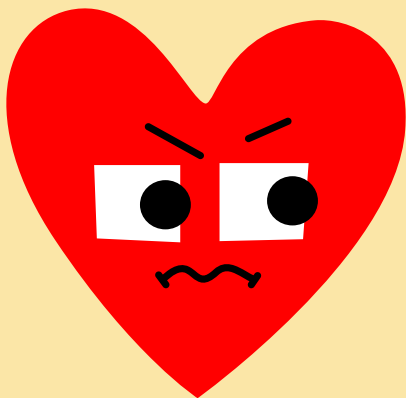
3. 数据的宽度和存储

3.1 大小端

大端存储：和人类阅读顺序一致，数据低位放在高地址，数据高位放在低地址

小端存储：和人类阅读顺序相反，数据低位放在低地址，数据高位放在高地址

图灵社区会员专享



一个编址单位内部的数据不受大小端影响

结构体内部的成员之间存放顺序不受大小端影响



3. 数据的宽度和存储

3.2 边界对齐

按x编址：存储器中每x位编一个地址，即一个地址中存储x位

0800H	0801H	0802H	0803H
0000 1101	1011 0101	1111 1110	1100 1010

按字节编址

0800H	0801H	0802H	0803H
FFFE 0101H	EEBA A889H	7774 BA8DH	FF66 8800H

32位的机器，按**字**编址



3. 数据的宽度和存储

3.2 边界对齐

k 字节大小的数据起始地址必须是 k 的整数倍

32位的机器
按字节编址

0800H	0801H	0802H	0803H
x1	x1		
0804H	0805H	0806H	0807H
x2	x2	x2	x2

```
struct record{  
    short x1;  
    int x2;  
}
```

边界对齐可以保证以最快速度读写数据



3. 数据的宽度和存储

3.2 边界对齐

k 字节大小的数据起始地址必须是 k 的整数倍

32位的机器
按**字节**编址

0800H	0801H	0802H	0803H
x1	x1	x2	x2
0804H	0805H		
x2	x2		

此时需要2次访问主存
才能将x2取出

```
struct record{  
    short x1;  
    int x2;  
}
```



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

1. 【2020统考真题】在按字节编址、采用小端方式的32位计算机中，按边界对齐方式为以下C语言结构型变量a分配存储空间：



```
struct record{  
    short x1;  
    int x2;  
} a;
```

若a的首地址是2020 FE00H，a的成员变量x2的机器数为1234 0000H，则其中34H所在存储单元的地址是

A.2020 FE03H

B.2020 FE04H

C.2020 FE05H

D.2020 FE06H